GOLDEN COLLAR INSTITUTE

Projet Data Engineering — Apache Spark & Data Visualization

Système de Gestion de l'Émargement

Intelligence Artificielle Intégrée

Année 2025–2026 | Data Engineering

Table des matières

1 Vue d'ensemble du projet **2**
- 1.1 Objectif général . . . 2
- 1.2 Sessions couvertes . . . 2
- 1.3 Stack technologique . . . 2
- 1.4 Architecture globale du pipeline . . . 2

2 Structure du projet (GitHub) **3**

3 Modèle de données PostgreSQL **3**
- 3.1 Schéma `ref` — Données de référence . . . 3
  - 3.1.1 Tables du schéma `ref` . . . 4
  - 3.1.2 Détail des tables et colonnes . . . 4
  - 3.1.3 Diagramme relationnel du schéma `ref` . . . 5
- 3.2 Schéma `bronze` — Données brutes . . . 5
- 3.3 Schéma `silver` — Données nettoyées . . . 5
- 3.4 Schéma `gold` — Indicateurs agrégés . . . 6

4 Plan de travail détaillé — Étapes du pipeline **6**
- 4.1 Phase 0 — Mise en place de l'environnement . . . 6
- 4.2 Phase 1 — Generate (Génération des feuilles Excel) . . . 7
- 4.3 Phase 2 — Collect (Récupération des feuilles remplies) . . . 7
- 4.4 Phase 3 — Extract (Couche Bronze) . . . 8
- 4.5 Phase 4 — Transform (Couche Silver) . . . 8
- 4.6 Phase 5 — Aggregate (Couche Gold) . . . 9
- 4.7 Phase 6 — Dashboard (Streamlit) . . . 10

5 Orchestration Apache Airflow **10**
- 5.1 Structure des DAGs . . . 10
- 5.2 Enchaînement des tâches . . . 10
- 5.3 Détail des tâches Airflow . . . 11

6 Couche Intelligence Artificielle (Bonus) **11**
- 6.1 Module 1 — Prédiction des absences . . . 11
- 6.2 Module 2 — Détection d'anomalies . . . 12
- 6.3 Module 3 — Scoring étudiant . . . 12
- 6.4 Intégration dans le pipeline Airflow . . . 12

7 Critères d'évaluation **13**

8 Livrables attendus **13**

9 Bonnes pratiques et points de vigilance **14**

1. Vue d'ensemble du projet

1.1. Objectif général

Le projet consiste à concevoir et implémenter un **pipeline de données automatisé** pour la gestion de l'émargement du Golden Collar Institute. Le système couvre l'intégralité du cycle : de la génération des feuilles de présence Excel jusqu'à l'affichage d'indicateurs en temps quasi-réel, enrichi d'une **couche IA** pour la détection d'anomalies et la prédiction des absences.

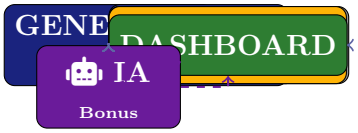
1.2. Sessions couvertes

Session	Génération Excel	Récupération	Jours
Matin	07h00	10h00	Lun.–Ven.
Après-midi	13h00	16h00	Lun.–Ven.

1.3. Stack technologique

Composant	Technologie
Traitement données	`Apache Spark (PySpark)`
Orchestration	`Apache Airflow`
Base de données	`PostgreSQL` (schémas bronze / silver / gold / ref)
Génération Excel	`openpyxl`
Dashboard	`Streamlit`
Machine Learning	`scikit-learn` , `PySpark MLlib`
Conteneurisation	`Docker` + `docker-compose`
Versioning	`Git` / `GitHub`

1.4. Architecture globale du pipeline



2. Structure du projet (GitHub)

```
emargement-project/
|
+-- dags/
|   +-- dag_matin.py            # DAG session matin (07h00)
|   +-- dag_apresmidi.py        # DAG session apres-midi (13h00)
|
+-- jobs/
|   +-- generate.py             # Generation des fichiers Excel
|   +-- collect.py              # Recuperation et verification
|   +-- extract.py              # Ingestion brute -> Bronze (PySpark)
|   +-- transform.py            # Nettoyage / enrichissement -> Silver
|   +-- aggregate.py            # Calcul indicateurs -> Gold
|   +-- ia/
|       +-- predict_absences.py # Modele de prediction
|       +-- detect_anomalies.py # Detection d'anomalies
|       +-- scoring.py          # Scoring etudiant
|
+-- database/
|   +-- schema.sql              # Creation schemas + tables
|   +-- seed_data.sql           # Donnees de test
|
+-- dashboard/
|   +-- app.py                  # Application Streamlit
|
+-- data/
|   +-- generated/              # Fichiers Excel generes
|   +-- collected/              # Fichiers Excel retournes
|
+-- utils/
|   +-- db.py                   # Connexion PostgreSQL
|   +-- logger.py               # Logging centralise
|
+-- models/                     # Modeles ML serialises (.pkl)
+-- .env                        # Credentials (jamais commite)
+-- .env.example                # Template .env
+-- docker-compose.yml
+-- README.md
```

3. Modèle de données PostgreSQL

La base de données est organisée en **quatre schémas** distincts. Le schéma `ref` repose sur le modèle relationnel suivant, adapté à la structure réelle des données.

3.1. Schéma `ref` — Données de référence

3.1.1. Tables du schéma `ref`

Table	Clé primaire	Description
`Etudiant`	`Numero_et (PK)`	Identité de l'étudiant, filière, niveau
`Enseignant`	`Id (PK)`	Identité de l'enseignant
`Cours`	`Id (PK)`	Intitulé, section, horaire, session, enseignant responsable
`Planning`	`Id (PK)`	Planification des cours : jour, heure de début/fin
`Inscription`	`(Id_etudiant, Cours_id)`	Relation étudiant ↔ cours + date d'inscription

3.1.2. Détail des tables et colonnes

`Numero_et`	PK	Identifiant unique de l'étudiant
`Nom`		Nom de famille
`Prenom`		Prénom
`Filiere`		Filière de rattachement
`Niveau`		Niveau d'études

`Id`	PK	Identifiant unique de l'enseignant
`Nom`		Nom de famille
`Prenom`		Prénom

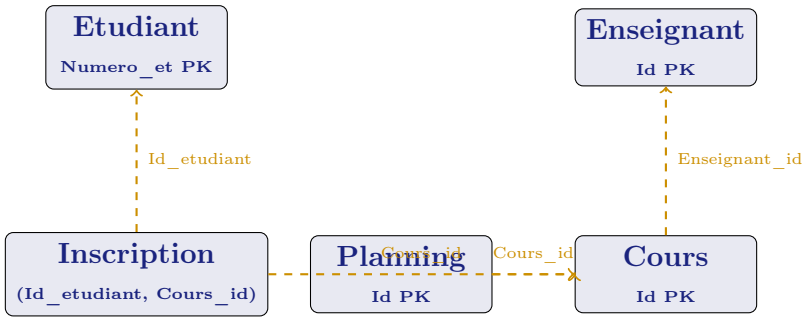
`Id`	PK	Identifiant unique du cours
`Intitule`		Nom du cours
`Section`		Section / groupe
`Horaire`		Créneau horaire (ex. 08h00–10h00)
`Session`		matin / après-midi
`Enseignant_id`	FK → `Enseignant.Id`	Enseignant responsable du cours

`Id`	PK	Identifiant unique du créneau
`Cours_id`	FK → `Cours.Id`	Cours planifié
`Jour_semaine`		Jour de la semaine (ex. Lundi)
`heure_debut`		Heure de début du cours
`heure_fin`		Heure de fin du cours

`Id_etudiant`	FK → `Etudiant.Numero_et`	Étudiant inscrit
`Cours_id`	FK → `Cours.Id`	Cours concerné
`Date_inscription`		Date d'inscription au cours

Clé primaire composite : `(Id_etudiant, Cours_id)`

3.1.3. Diagramme relationnel du schéma `ref`



3.2. Schéma `bronze` — Données brutes

`date`	Date de la session
`session`	matin / après-midi
`cours_id`	Identifiant du cours (depuis nom de fichier)
`etudiant_nom`	Nom brut (tel que saisi dans Excel)
`etudiant_prenom`	Prénom brut
`present`	Valeur brute (O / N / oui / 1 ...)
`remarque`	Remarque optionnelle
`fichier_source`	Nom du fichier Excel source

3.3. Schéma `silver` — Données nettoyées

`date`	Date normalisée
`session`	matin / après-midi
`cours_id`	FK → `ref.Cours.Id`
`numero_et`	FK → `ref.Etudiant.Numero_et`
`nom_complet`	Nom + prénom normalisés
`filiere`	Filière (issue de `ref.Etudiant`)
`niveau`	Niveau (issue de `ref.Etudiant`)
`present`	**boolean** (true / false)
`remarque`	Remarque nettoyée

3.4. Schéma `gold` — Indicateurs agrégés

`gold.presence_par_cours`	Taux de présence par cours, date, session
`gold.presence_par_etudiant`	Historique et taux cumulé par étudiant
`gold.absences_repetees`	Étudiants dépassant le seuil d'absences
`gold.retour_feuilles`	Taux de retour des feuilles par enseignant

4. Plan de travail détaillé — Étapes du pipeline

4.1. Phase 0 — Mise en place de l'environnement

Phase 0 — Setup & Infrastructure

Objectif : Préparer l'environnement de développement et les services nécessaires.

1. **Initialiser le dépôt GitHub**
   - Créer le dépôt avec `.gitignore` (exclure `.env`, `data/`)
   - Définir la structure de dossiers décrite section 2
   - Rédiger la première version du `README.md`
2. **Configurer Docker Compose**
   - Service `PostgreSQL` (port 5432)
   - Service `Apache Airflow` (webserver + scheduler)
   - Service `Apache Spark` (master + worker)
   - Fichier `.env.example` avec tous les paramètres nécessaires
3. **Créer les schémas PostgreSQL**
   - Rédiger `database/schema.sql` : schémas `ref`, `bronze`, `silver`, `gold`
   - Tables `ref` : `Etudiant`, `Enseignant`, `Cours`, `Planning`, `Inscription`
   - Contraintes FK : `Cours.Enseignant_id` → `Enseignant.Id`; `Planning.Cours_id` → `Cours.Id`; clé composite sur `Inscription`
   - Rédiger `database/seed_data.sql` : données de test (étudiants, cours, enseignants, planning, inscriptions)
4. **Utilitaires communs**
   - `utils/db.py` : connexion PostgreSQL via `psycopg2` / `SQLAlchemy`
   - `utils/logger.py` : logging structuré (niveau, timestamp, module)

Livrable : Dépôt GitHub initialisé, Docker Compose fonctionnel, BDD créée avec données de test.

4.2. Phase 1 — Generate (Génération des feuilles Excel)

Phase 1 — GENERATE (openpyxl)

Objectif : Générer automatiquement les feuilles d'émargement Excel.
Déclenchement : 07h00 (matin) / 13h00 (après-midi)

1. **Lecture des données de référence**
   - Requête `ref.Cours` + jointure `ref.Planning` : cours du jour filtrés sur la session courante (`Cours.Session` + `Planning.Jour_semaine`)
   - Requête `ref.Inscription JOIN ref.Etudiant` : liste des inscrits par cours (via `Inscription.Cours_id` et `Inscription.Id_etudiant = Etudiant.Numero_et`)
   - Jointure `ref.Cours JOIN ref.Enseignant` : récupérer le nom de l'enseignant responsable (`Cours.Enseignant_id = Enseignant.Id`)
2. **Génération des fichiers Excel avec openpyxl**
   - Un fichier par cours : `data/generated/YYYY-MM-DD/<session>/cours_<id>.xlsx`
   - Colonnes : **Numero_et**, **Nom**, **Prénom**, **Filière**, **Niveau**, **Présent (O/N)**, **Signature**, **Remarque**
   - En-tête avec : intitulé du cours, section, date, session, créneau horaire (`Planning.heure_debut`–`heure_fin`), enseignant responsable
   - Mise en forme : largeurs de colonnes, bordures, ligne d'en-tête colorée
3. **Dépôt des fichiers**
   - Création automatique de l'arborescence de dossiers si absente
   - Log du nombre de fichiers générés et des erreurs éventuelles

Point d'attention : Les credentials PostgreSQL doivent être lus depuis `.env` uniquement — jamais codés en dur dans le script.

4.3. Phase 2 — Collect (Récupération des feuilles remplies)

Phase 2 — COLLECT

Objectif : Scanner les dossiers et vérifier la complétude des fichiers retournés.
Déclenchement : 10h00 (matin) / 16h00 (après-midi)

1. **Scan du dossier de collecte**
   - Parcourir `data/collected/YYYY-MM-DD/<session>/`
   - Lister les fichiers présents et les comparer aux fichiers attendus
2. **Vérification de complétude**
   - Comparer la liste des fichiers trouvés avec les fichiers générés (Phase 1)
   - Détecter les fichiers manquants : rejoindre avec `ref.Cours JOIN ref.Enseignant` pour identifier l'enseignant n'ayant pas retourné sa feuille

— Détecter les fichiers vides ou corrompus

3. **Rapport et alertes**
   — Logger les fichiers manquants avec le nom et prénom de l'enseignant concerné
   — Mettre à jour `gold.retour_feuilles` (taux de retour en temps réel)
   — Continuer le pipeline avec les fichiers disponibles (pas de blocage total)

4.4. Phase 3 — Extract (Couche Bronze)

Phase 3 — EXTRACT → Bronze (PySpark)

Objectif : Ingérer les fichiers Excel bruts dans PostgreSQL, schéma `bronze`.

1. **Initialisation de la session Spark**
   — Configurer la SparkSession avec le connecteur JDBC PostgreSQL
   — Charger la dépendance `com.crealytics:spark-excel`
2. **Lecture des fichiers Excel avec PySpark**
   — Itérer sur la liste des fichiers collectés
   — Lire chaque fichier : `spark.read.format("excel").load(path)`
   — Valider le schéma minimum attendu (colonnes `Numero_et`, `Nom`, `Prenom`, `Present` présentes)
3. **Ajout des métadonnées**
   — `date` : date du jour
   — `session` : matin / après-midi (issue de `Cours.Session`)
   — `cours_id` : extrait du nom de fichier (correspond à `Cours.Id`)
   — `fichier_source` : chemin complet du fichier
4. **Écriture en base**
   — Mode `append` vers `bronze.emargement_raw` via JDBC
   — Logger le nombre de lignes ingérées et les fichiers en erreur

4.5. Phase 4 — Transform (Couche Silver)

Phase 4 — TRANSFORM → Silver (PySpark)

Objectif : Nettoyer, normaliser et enrichir les données brutes.

1. **Chargement depuis Bronze**
   — Lire `bronze.emargement_raw` pour la session et la date courantes
2. **Normalisation de la colonne `present`**
   — Standardiser : `O` / `oui` / `1` / `yes` → `true`
   — Standardiser : `N` / `non` / `0` / `no` → `false`
   — Valeurs inconnues → `null` + flag d'anomalie

3. **Déduplication**
   - Supprimer les doublons sur (`date`, `session`, `cours_id`, `etudiant_nom`, `etudiant_prenom`)
   - Logger les doublons détectés pour audit
4. **Jointure avec les données de référence**
   - Jointure avec `ref.Etudiant` sur `Nom` + `Prenom` : récupérer `Numero_et`, `Filiere`, `Niveau`
   - Signaler les étudiants non reconnus (présents dans Excel mais absents de `ref.Etudiant`)
   - Vérifier la cohérence avec `ref.Inscription` : l'étudiant doit être inscrit au cours (`Inscription.Id_etudiant = Numero_et` et `Inscription.Cours_id = cours_id`)
5. **Écriture Silver**
   - Mode `append` vers `silver.emargement_clean`

4.6. Phase 5 — Aggregate (Couche Gold)

Phase 5 — AGGREGATE → Gold (PySpark)

Objectif : Calculer les indicateurs métier et alimenter les tables analytiques.

1. **Taux de présence par cours**
   - $\text{taux} = \frac{\text{nb_presents}}{\text{nb_inscrits}} \times 100$
   - Nombre d'inscrits calculé depuis `ref.Inscription` (groupé par `Cours_id`)
   - Granularité : par `cours_id`, `date`, `session`
   - Cible : `gold.presence_par_cours`
2. **Historique de présence par étudiant**
   - Taux cumulé sur la semaine et le mois (groupé par `Numero_et`)
   - Enrichi avec `Filiere` et `Niveau` issus de `ref.Etudiant`
   - Cible : `gold.presence_par_etudiant`
3. **Détection des absences répétées**
   - Identifier les étudiants dépassant le seuil N d'absences consécutives
   - Seuil configurable dans `.env`
   - Cible : `gold.absences_repetees`
4. **Taux de retour des feuilles enseignants**
   - Ratio fichiers reçus / fichiers attendus par enseignant (jointure `ref.Cours JOIN ref.Enseignant` via `Cours.Enseignant_id`)
   - Cible : `gold.retour_feuilles`

4.7. Phase 6 — Dashboard (Streamlit)

Phase 6 — Dashboard Streamlit

Objectif : Visualiser les indicateurs en temps quasi-réel depuis les tables Gold.

1. **Connexion PostgreSQL**
   - Lecture directe des tables `gold.*` via `SQLAlchemy` + `pandas`
   - Rafraîchissement automatique toutes les N minutes (`st.rerun`)
2. **Indicateurs obligatoires**
   - Taux de présence global du jour (matin + après-midi)
   - Liste des étudiants absents du jour (avec `Filiere` et `Niveau`)
   - Évolution du taux de présence sur la semaine (graphique)
   - Top des étudiants les plus absents
   - Alertes : étudiants ayant dépassé le seuil d'absences
   - Taux de retour des feuilles par enseignant (Nom + Prénom)
3. **Indicateurs IA (bonus dashboard)**
   - Prédictions d'absence du lendemain
   - Anomalies détectées (absences suspectes)
   - Score de sérieux par étudiant avec badge coloré
   - Filtres interactifs : filière, niveau, cours, section, période

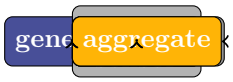
Justification Streamlit : Prototypage rapide, connexion native PostgreSQL via pandas, déploiement simplifié. Adapté à une démo live en contexte académique.

5. Orchestration Apache Airflow

5.1. Structure des DAGs

DAG	Schedule (cron)	Jours
`emargement_matin`	`0 7 * * 1-5`	Lundi–Vendredi
`emargement_apresmidi`	`0 13 * * 1-5`	Lundi–Vendredi

5.2. Enchaînement des tâches



5.3. Détail des tâches Airflow

1. **Task** `generate` : `PythonOperator` → appel de `jobs/generate.py`
2. **Task** `wait_sensor` : `TimeDeltaSensor` ou `FileSensor`
   — Attendre la disponibilité des fichiers entre génération et collecte
   — Timeout configurable, retry automatique
3. **Task** `collect` : `PythonOperator` → `jobs/collect.py`
4. **Tasks** `extract`, `transform`, `aggregate` : `SparkSubmitOperator`
   — Paramètres : date, session passés via `op_kwargs`
5. **Gestion des erreurs**
   — `retries=2`, `retry_delay=timedelta(minutes=5)`
   — Email d'alerte en cas d'échec (`email_on_failure=True`)
   — Chaque task loggue son statut dans `ref.pipeline_logs`

6. Couche Intelligence Artificielle (Bonus)

Couche IA — Niveau Data Scientist

Objectif : Enrichir le pipeline avec des modèles prédictifs et de détection d'anomalies, exploitant les données agrégées de la couche Gold pour produire des insights à haute valeur ajoutée.

6.1. Module 1 — Prédiction des absences

1. **Collecte des features**
   — Historique d'absences par étudiant (`gold.presence_par_etudiant`)
   — Jour de la semaine (`Planning.Jour_semaine`), créneau horaire (`Planning.heure_debut`), nombre de cours dans la journée
   — Filière et niveau (`Etudiant.Filiere`, `Etudiant.Niveau`), absences récentes sur les N derniers jours
   — Section du cours (`Cours.Section`)
2. **Entraînement du modèle**
   — Baseline : `Logistic Regression` (interprétable, rapide)
   — Modèle avancé : `Random Forest` ou `Gradient Boosting`
   — Librairie : `scikit-learn` (ou `PySpark MLlib` pour le passage à l'échelle)
   — Variable cible : `present` (booléen)
   — Métriques : précision, rappel, F1-score, AUC-ROC
3. **Inférence quotidienne**
   — Prédire les absences du lendemain pour chaque étudiant/cours en consultant `ref.Planning` pour les cours du lendemain
   — Stocker dans `gold.predictions_absences`
   — Afficher dans le dashboard : *Étudiants à risque demain*

6.2. Module 2 — Détection d'anomalies

1. **Absences suspectes**
   - Détecter les patterns inhabituels : absent uniquement certains jours (`Planning.Jour_semaine`), absent avec un seul enseignant, présent à un seul cours sur cinq ...
   - Algorithme : `Isolation Forest` ou règles métier paramétrables
2. **Incohérences dans les feuilles**
   - Étudiant présent dans Excel mais absent de `ref.Inscription` (pas inscrit au cours)
   - Taux de présence anormalement bas ou élevé pour un cours donné
3. **Stockage**
   - Table `gold.anomalies_detectees` : type, `Numero_et`, `cours_id`, date, score d'anomalie

6.3. Module 3 — Scoring étudiant

1. **Calcul du score de sérieux**

   $$\text{score} = \alpha \times \text{taux_presence} + \beta \times \text{reg_retour_feuilles} + \gamma \times (1 - \text{taux_absences_consécutives})$$

   avec $\alpha + \beta + \gamma = 1$ (poids configurables dans `.env`)
2. **Classification en trois niveaux**

Score	Label	Badge	Action déclenchée
$\geq 80\%$	Sérieux	✅	Aucune
50–79%	Moyen	⚠	Alerte préventive
$< 50\%$	À risque	❌	Alerte critique + convocation

3. **Stockage et affichage**
   - Table `gold.scoring_etudiants` avec `Numero_et`, `Nom`, `Prenom`, `Filiere`, `Niveau`, score, badge (mis à jour quotidiennement)
   - Dashboard : vue classement avec badge coloré et évolution du score

6.4. Intégration dans le pipeline Airflow

Les tasks IA sont ajoutées **en aval de** `aggregate` dans chaque DAG :

Chaque module IA est un `PythonOperator` utilisant les tables Gold comme source et écrivant dans des tables Gold dédiées (`gold.predictions_absences`, `gold.anomalies_detectees`, `gold.scoring_etudiants`).

7. Critères d'évaluation

Critère	Phases	Points	Priorité
Génération et collecte Excel	1 + 2	/3	Haute
Pipeline Spark fonctionnel	3 + 4 + 5	/3	Haute
Modèle PostgreSQL (bronze/silver/gold)	0 + 3–5	/3	Haute
Orchestration Airflow	DAGs	/3	Haute
Dashboard et indicateurs	6	/4	Haute
DAT complété	Docs	/2	Moyenne
Qualité du code (GitHub)	Tous	/2	Moyenne
Couche IA (Bonus)	IA	+points	Bonus
Total		**/20**	

8. Livrables attendus

L1. Code source (GitHub)

- Scripts `generate.py`, `collect.py`, `extract.py`, `transform.py`, `aggregate.py`
- Dossier `jobs/ia/` : modules de prédiction, détection, scoring
- DAGs Airflow : `dag_matin.py`, `dag_apresmidi.py`
- `docker-compose.yml`, `.env.example`, `README.md` complet avec screenshots

L2. Scripts SQL

- `database/schema.sql` : création des 4 schémas et toutes les tables, notamment `ref.Etudiant`, `ref.Enseignant`, `ref.Cours`, `ref.Planning`, `ref.Inscription` (avec clé primaire composite et contraintes FK)
- `database/seed_data.sql` : données de référence suffisantes pour les tests (étudiants avec filière/niveau, enseignants, cours avec section/horaire/session, plannings, inscriptions)

L3. DAT — Document d'Architecture Technique

- Description de l'architecture, des choix technologiques et leur justification
- Schéma du pipeline, modèle de données (diagramme ERD `ref`), diagramme Airflow
- Section IA : modèles utilisés, features, métriques, résultats obtenus

L4. Démo live

- Simulation d'une journée complète : session matin + session après-midi
- Affichage en direct du dashboard avec indicateurs mis à jour
- Démonstration des alertes et des prédictions / scores IA

9. Bonnes pratiques et points de vigilance

- Toujours charger les credentials depuis `.env` — jamais dans le code source
- Commits atomiques et messages de commit descriptifs
- Chaque fonction documentée avec sa signature et son rôle
- Tests unitaires sur les fonctions de normalisation (Transform)
- Logger systématiquement : nombre de lignes traitées, erreurs, durée
- Gérer le mode `idempotent` : relancer un job ne doit pas dupliquer les données

Pièges fréquents à éviter :

- Ne pas gérer les fichiers Excel vides ou mal formés dans `extract.py`
- Oublier le mode `append` vs `overwrite` en écriture Spark/JDBC
- Committer le fichier `.env` sur GitHub (vérifier le `.gitignore`)
- Ne pas tester le pipeline sur des données de test avant la démo live
- Entraîner les modèles IA avec trop peu de données (utiliser les données de seed)
- Oublier la contrainte de clé primaire composite sur `ref.Inscription` lors des insertions en mode `append`